

Phase 2 - Local Quiz Engine and Score Persistence

Phase objective

The objective of Phase 2 was to transform the Quiz Learning Platform from a static React shell into a complete, playable local quiz experience before connecting the product to any backend or AI provider.

At the end of this phase, the application includes:

- A reusable quiz domain model
- Multiple-choice questions
- True-or-false questions
- Short-answer questions
- Deterministic answer validation
- Normalized short-answer comparison
- Correct and incorrect feedback
- Required answer explanations
- Question navigation
- Submitted-answer locking
- Final score calculation
- A quiz result screen
- Scoreboard updates after quiz completion
- Browser persistence with `localStorage`
- Component, utility, integration, and persistence tests

The quiz is still generated from deterministic fake data. This is intentional: the frontend behavior and domain rules are now stable before introducing the Java and Spring Boot backend.

Product decisions made during this phase

Support three question types

The quiz engine supports exactly three question types:

```
export type QuizQuestionType =  
  | "multiple-choice"  
  | "true-false"  
  | "short-answer";
```

Multiple-choice and true-or-false questions share the same option-based interaction. Short-answer questions use text input and accepted-answer comparison.

Explanations are required

Every question requires an explanation. This is part of the product's learning value, not optional presentation content.

```
interface BaseQuizQuestion {  
  id: string;  
  prompt: string;  
  explanation: string;  
}
```

Making `explanation` required means TypeScript rejects incomplete questions during development.

Grade short answers deterministically

The first version does not call an AI model to grade short answers. It compares normalized user input with configured accepted answers.

This keeps grading:

- Fast
- Predictable
- Testable
- Free of API cost
- Available offline

AI-assisted semantic grading can be evaluated later as an optional backend capability.

Lock submitted answers

After a learner checks an answer, its input becomes disabled. This prevents changing the answer after seeing the correct result.

Select answer -> Check answer -> Lock input -> Show feedback

Persist only completed scores

The application persists completed quiz results, not every temporary interaction. Current question state and text input remain in React memory. Completed scores are stored in `localStorage`.

Keep the quiz provider replaceable

Phase 2 uses `fakeQuiz`, but the frontend depends on a `Quiz` domain model rather than on the fake data source. The next phase can replace the fake provider with a Spring Boot REST API without rewriting the quiz components.

Final feature structure

```
apps/web/src/  
|-- features/  
|   |-- quiz-game/  
|   |   |-- fakeQuiz.ts  
|   |   |-- OptionQuestion.tsx  
|   |   |-- OptionQuestion.test.tsx  
|   |   |-- ShortAnswerQuestion.tsx  
|   |   |-- ShortAnswerQuestion.test.tsx  
|   |   |-- QuizGameScreen.tsx  
|   |   |-- QuizGameScreen.test.tsx  
|   |   |-- QuizResultScreen.tsx  
|   |   |-- QuizResultScreen.test.tsx  
|   |   |-- quiz.types.ts  
|   |   |-- quizAnswer.utils.ts  
|   |   |-- `-- quizAnswer.utils.test.ts  
|   |-- `-- scores/  
|   |   |-- scoreStorage.ts  
|   |   |-- `-- scoreStorage.test.ts  
|-- App.css  
|-- App.test.tsx  
|-- App.tsx
```

The exact filename capitalization must match its imports. The preferred React convention is `QuizGameScreen.tsx` because the component is named `QuizGameScreen`.

Quiz domain model

Base question data

All questions share identity, visible prompt, and explanation:

```
interface BaseQuizQuestion {  
  id: string;  
  prompt: string;  
  explanation: string;  
}
```

Answer options

Option-based questions use stable IDs:

```
export interface QuizAnswerOption {  
  id: string;
```

```

    text: string;
}

```

The ID is the stable identity. The text is presentation data and may later change or be translated.

Option-based questions

Multiple-choice and true-or-false questions share one structure:

```

export interface OptionQuizQuestion
  extends BaseQuizQuestion {
  type: "multiple-choice" | "true-false";
  options: QuizAnswerOption[];
  correctOptionId: string;
}

```

True-or-false is modeled with two normal options:

```

[
  { id: "true", text: "True" },
  { id: "false", text: "False" },
]

```

Short-answer questions

Short-answer questions do not have selectable options:

```

export interface ShortAnswerQuizQuestion
  extends BaseQuizQuestion {
  type: "short-answer";
  acceptedAnswers: string[];
}

```

The array allows more than one valid representation.

Discriminated union

The final question type is a discriminated union:

```

export type QuizQuestion =
  | OptionQuizQuestion
  | ShortAnswerQuizQuestion;

```

The type property tells TypeScript which complete object shape is active.

```

if (question.type === "short-answer") {
  question.acceptedAnswers;
} else {
  question.options;
}

```

```
    question.correctOptionId;
}
```

This is stronger than only adding another string to `QuizQuestionType`. The object structure changes with the question type.

Quiz and answer models

```
export interface Quiz {
  id: string;
  title: string;
  difficulty: QuizDifficulty;
  questions: QuizQuestion[];
}
```

A user answer is also a discriminated union:

```
export type QuizAnswer =
  | {
    questionId: string;
    kind: "option";
    selectedOptionId: string;
  }
  | {
    questionId: string;
    kind: "text";
    value: string;
  };
};
```

The question defines the validation rules. The answer represents what the user submitted.

Fake quiz data

`fakeQuiz.ts` provides deterministic data while the backend does not exist.

The fake quiz includes:

- One multiple-choice question
- One true-or-false question
- One short-answer question
- Required explanations for all questions

Fake data enabled the team to build and test the complete product flow before paying the complexity cost of API communication or AI integration.

Fake quiz -> Local engine -> Stable frontend contract -> Backend later

Pure answer validation

Answer validation is domain logic, not rendering logic. It lives in `quizAnswer.utils.ts`.

Short-answer normalization

```
export function normalizeShortAnswer(  
  value: string,  
): string {  
  return value  
    .trim()  
    .toLowerCase()  
    .replace(/\s+/g, " ");  
}
```

The following inputs are treated as equivalent:

GROUP BY
group by
Group By

Normalization currently:

- Removes leading and trailing whitespace
- Converts text to lowercase
- Collapses repeated whitespace

Accents are not automatically removed. For Portuguese questions, accepted spelling variants should be configured deliberately because accents may be meaningful.

Runtime answer validation

```
export function isQuizAnswerCorrect(  
  question: QuizQuestion,  
  answer: QuizAnswer,  
): boolean {  
  if (question.id !== answer.questionId) {  
    return false;  
  }  
  
  if (question.type === "short-answer") {  
    if (answer.kind !== "text") {  
      return false;  
    }  
  
    const normalizedAnswer = normalizeShortAnswer(  
      answer.value,
```

```

    );

    return question.acceptedAnswers.some(
      (acceptedAnswer) =>
        normalizeShortAnswer(acceptedAnswer) ===
        normalizedAnswer,
    );
  }

  if (answer.kind !== "option") {
    return false;
  }

  return (
    question.correctOptionId ===
    answer.selectedOptionId
  );
}

```

Important protections include:

- Matching `questionId`
- Matching question type with answer kind
- Comparing option IDs rather than visible text
- Normalizing text before comparison

Correct answer text

`getCorrectAnswerText` converts validation data into user-facing feedback:

```

export function getCorrectAnswerText(
  question: QuizQuestion,
): string {
  if (question.type === "short-answer") {
    return question.acceptedAnswers[0] ??
      "No accepted answer was configured.";
  }

  const correctOption = question.options.find(
    (option) =>
      option.id === question.correctOptionId,
  );

  return correctOption?.text ??
    "The correct answer is unavailable.";
}

```

Controlled question components

OptionQuestion

OptionQuestion receives:

```
interface OptionQuestionProps {
  question: OptionQuizQuestion;
  selectedOptionId: string | null;
  disabled: boolean;
  onSelectOption: (optionId: string) => void;
}
```

It does not use local `useState`. The parent owns the selected option.

Parent state

-> selectedOptionId prop

OptionQuestion

-> onSelectOption callback

Parent updates state

All radio inputs use the same `name` so the browser treats them as one group.
Each input uses a unique `id` so its label targets the correct control.

ShortAnswerQuestion

ShortAnswerQuestion receives:

```
interface ShortAnswerQuestionProps {
  question: ShortAnswerQuizQuestion;
  value: string;
  disabled: boolean;
  onChange: (value: string) => void;
}
```

It is also controlled. The parent owns the text and passes the updated value back after each change.

Why controlled components were chosen

Controlled components provide:

- One source of truth
- Predictable input state
- Easy reset between questions
- Easy answer locking
- Simple integration testing
- Reusable presentational components

Application screen state

The application originally used a boolean for two screens. Phase 2 introduced a third screen, so a string union became clearer:

```
type AppScreen =  
  | "home"  
  | "quiz-configuration"  
  | "quiz-game";
```

This avoids invalid combinations that multiple booleans could represent.

One union value -> exactly one active screen

React Router is still not required because the product does not yet require deep links, shareable quiz URLs, or browser-history navigation.

Quiz game state ownership

QuizGameScreen owns the active quiz session.

Its state includes:

```
const [currentQuestionIndex, setCurrentQuestionIndex] =  
  useState(0);  
  
const [selectedOptionId, setSelectedOptionId] =  
  useState<string | null>(null);  
  
const [shortAnswerValue, setShortAnswerValue] =  
  useState("");  
  
const [isAnswerSubmitted, setIsAnswerSubmitted] =  
  useState(false);  
  
const [correctAnswerCount, setCorrectAnswerCount] =  
  useState(0);  
  
const [isQuizComplete, setIsQuizComplete] =  
  useState(false);
```

Current question

```
const currentQuestion =  
  quiz.questions[currentQuestionIndex];
```

Array indexes begin at zero, while visible question numbers begin at one:

```

    Question {currentQuestionIndex + 1} of{
      quiz.questions.length
    }
  }

```

Derived values

Values such as the current question, last-question status, answer object, and correctness are derived from state and props.

```

const isLastQuestion =
  currentQuestionIndex === quiz.questions.length - 1;

```

Correctness is derived from the current question and current answer:

```

const isCorrect =
  isAnswerSubmitted && answer
    ? isQuizAnswerCorrect(currentQuestion, answer)
    : null;

```

The three possible values are meaningful:

```

null   -> not checked yet
true   -> correct
false  -> incorrect

```

Question rendering

The discriminated union controls which component is rendered:

```

{currentQuestion.type === "short-answer" ? (
  <ShortAnswerQuestion
    question={currentQuestion}
    value={shortAnswerValue}
    disabled={isAnswerSubmitted}
    onChange={handleShortAnswerChange}
  />
) : (
  <OptionQuestion
    question={currentQuestion}
    selectedOptionId={selectedOptionId}
    disabled={isAnswerSubmitted}
    onSelectOption={handleSelectOption}
  />
)}

```

TypeScript narrows `currentQuestion` automatically inside each branch.

Submission and feedback

Defensive submission handler

```
function handleCheckAnswer(): void {
  if (!answer || isAnswerSubmitted) {
    return;
  }

  const answerIsCorrect = isQuizAnswerCorrect(
    currentQuestion,
    answer,
  );

  if (answerIsCorrect) {
    setCorrectAnswerCount(
      (currentCount) => currentCount + 1,
    );
  }

  setIsAnswerSubmitted(true);
}
```

The disabled button protects the UI. The early return protects the function.

UI protection -> disabled button

Logic protection -> guard clause

The previous-state callback is used because the new score depends on the previous score.

Feedback

After submission, the interface displays:

- Correct or incorrect result
- Correct answer when the learner was wrong
- Required explanation in both cases

The feedback section uses:

`aria-live="polite"`

This allows assistive technology to announce dynamic feedback without interrupting the learner aggressively.

Answer locking

Submitted inputs receive:

```
disabled={isAnswerSubmitted}
```

This prevents a learner from seeing the answer, editing the response, and submitting again.

The submission handler also guards against duplicate counting:

```
if (!answer || isAnswerSubmitted) {  
  return;  
}
```

The UI and business logic both enforce the rule.

Question navigation

```
function handleNextQuestion(): void {  
  const nextQuestionIndex = currentQuestionIndex + 1;  
  
  if (nextQuestionIndex >= quiz.questions.length) {  
    return;  
  }  
  
  setCurrentQuestionIndex(nextQuestionIndex);  
  setSelectedOptionId(null);  
  setShortAnswerValue("");  
  setIsAnswerSubmitted(false);  
}
```

Navigation resets all question-specific input and feedback state.

Next question

- > increment index
- > clear option selection
- > clear text answer
- > clear submission status

Without these resets, the next question could display stale data from the previous question.

Quiz completion and result screen

Completion result model

```
export interface QuizCompletionResult {  
  quizTitle: string;  
  difficulty: QuizDifficulty;  
  correctAnswers: number;
```

```

    totalQuestions: number;
  }

```

This model intentionally excludes persistence fields such as `id` and `completedAt`. The quiz reports what happened; the application boundary decides how to store it.

Finish handler

```

function handleFinishQuiz(): void {
  onComplete({
    quizTitle: quiz.title,
    difficulty: quiz.difficulty,
    correctAnswers: correctAnswerCount,
    totalQuestions: quiz.questions.length,
  });

  setIsQuizComplete(true);
}

```

Result component

`QuizResultScreen` is presentational. It receives counts and derives the percentage through the existing score utility.

```

const percentage = calculateScorePercentage({
  correctAnswers,
  totalQuestions,
});

```

The result screen does not decide which answers were correct. `QuizGameScreen` owns that behavior.

```

QuizGameScreen -> session rules and score count
QuizResultScreen -> final presentation

```

Updating the home scoreboard

`App` owns the collection of completed scores.

```

const [scores, setScores] = useState<QuizScore[]>(...);

```

When a quiz is completed, `App` creates persistence-specific fields:

```

const completedScore: QuizScore = {
  id: crypto.randomUUID(),
  quizTitle: result.quizTitle,
  difficulty: result.difficulty,
}

```

```

    correctAnswers: result.correctAnswers,
    totalQuestions: result.totalQuestions,
    completedAt: new Date().toISOString(),
  };

```

It then adds the newest score first:

```

setScores((currentScores) => [
  completedScore,
  ...currentScores,
]);

```

HomeScreen receives `scores` through props, so it did not need to know whether scores came from mock data, React state, or browser persistence.

Browser persistence boundary

Why storage logic is separate

`scoreStorage.ts` isolates browser persistence from React components.

App -> owns running state

scoreStorage -> reads and writes localStorage

The utility does not call React state setters.

Storage key

```

const QUIZ_SCORES_STORAGE_KEY =
  "quiz-learning-platform:scores";

```

A namespaced key reduces the risk of collisions with unrelated browser data.

Runtime validation

localStorage contains strings, and `JSON.parse` can produce any JSON shape. TypeScript types do not validate runtime data.

```

const parsedScores: unknown = JSON.parse(storedScores);

```

The value remains `unknown` until validation proves it is a `QuizScore[]`.

```

function isQuizScore(value: unknown): value is QuizScore {
  // Runtime field validation
}

```

This is safer than:

```

JSON.parse(storedScores) as QuizScore[];

```

A type assertion would only silence TypeScript; it would not validate the data.

Fallback behavior

```
export function loadQuizScores(  
  fallbackScores: QuizScore[] = [],  
) : QuizScore[] {  
  // Return stored valid scores or fallback scores.  
}
```

Fallback scores give the application valid initial data when storage is empty, corrupted, or unavailable.

Error handling

try/catch protects the app from:

- Invalid JSON
- Storage restrictions
- Browser privacy settings
- Quota failures
- Unexpected runtime values

A storage failure should not crash the quiz. React can continue using in-memory state.

Connecting persistence to React

Lazy state initializer

```
const [scores, setScores] = useState<QuizScore[]>(  
  () => loadQuizScores(mockScores),  
);
```

Passing a function to `useState` means React loads storage only when the state is first created.

Initial render -> read `localStorage`

Later renders -> reuse React state

Persisting changes with `useEffect`

```
useEffect(() => {  
  saveQuizScores(scores);  
}, [scores]);
```

The effect runs after rendering and whenever the `scores` reference changes.

While the application is running, React state is the source of truth. `localStorage` is the persistence mechanism used to restore state after a page reload.

Accessibility decisions

Phase 2 preserves semantic and accessible interaction:

- `fieldset` and `legend` group each question
- Labels connect to inputs through `htmlFor` and `id`
- Radio groups share a `name`
- Text inputs have visible labels
- Disabled controls use native `disabled`
- Feedback uses `aria-live="polite"`
- Tests query elements by role and accessible name

Using native browser semantics reduces the amount of custom accessibility behavior required.

Testing strategy

Pure utility tests

`quizAnswer.utils.test.ts` verifies:

- Short-answer normalization
- Correct option validation
- Incorrect option rejection
- True-or-false validation
- Question ID mismatch rejection
- Correct answer text lookup

`scoreStorage.test.ts` verifies:

- Empty storage fallback
- Saving and loading scores
- Invalid JSON fallback
- Invalid object-shape fallback

Presentational component tests

`OptionQuestion.test.tsx` verifies:

- Prompt and options render
- Callback receives the selected option ID
- Provided selection appears checked
- Inputs become disabled after submission

`ShortAnswerQuestion.test.tsx` verifies:

- Prompt and textbox render

- Parent-controlled value appears
- Typing is reported to the parent
- Textbox becomes disabled after submission

`QuizResultScreen.test.tsx` verifies:

- Title and final percentage render
- Summary text includes correct counts
- Return-home callback runs

Quiz integration tests

`QuizGameScreen.test.tsx` verifies the complete short-answer flow:

```
Type answer
-> Check answer
-> Show correct feedback
-> Finish quiz
-> Report completion
-> Display result
-> Return home
```

Application integration tests

`App.test.tsx` verifies behavior across screens:

- Configuration starts a quiz
- Selecting an answer enables submission
- Navigation moves to the next question
- Completing a quiz updates the scoreboard
- Completed scores are persisted

Test isolation

`localStorage` is cleared before each application test:

```
beforeEach(() => {
  window.localStorage.clear();
});
```

This prevents one test from leaking saved scores into another test.

Problems encountered and lessons learned

Inconsistent filename capitalization

The project temporarily used both:

```
QuizGameScreen.tsx
quizGameScreen.tsx
```

Windows often treats them as the same path, while TypeScript, Git, Linux, and CI may enforce exact casing.

Lesson: choose one filename and make every import match it exactly.

Duplicate Check answer buttons

A conditional action button was placed inside the header while another action button remained below the question. React Testing Library found two buttons with the same accessible name.

Lesson: when a semantic query finds duplicates, inspect the rendered UI before weakening the test with `getAllByRole`.

Temporary short-answer early return remained

The first short-answer implementation returned a temporary message before the normal quiz UI. The new short-answer test could not find the textbox or submission button.

Lesson: remove temporary control flow when the real implementation replaces it.

Incorrect import or export name

The component referenced `getCorrectAnswerText`, while an earlier import used `getCorrectAnswer`.

Lesson: function names in imports and exports must match exactly; descriptive names reduce ambiguity.

Wrong function signature

`calculateScorePercentage` accepts one score-like object, not two positional arguments.

Correct:

```
calculateScorePercentage({
  correctAnswers,
  totalQuestions,
});
```

Lesson: inspect the function contract instead of assuming a familiar signature.

Text assertion split by nested elements

The result summary used nested **strong** elements, so a single `getByText` matcher could not find the complete sentence as one text node.

The improved test finds the parent paragraph and checks:

```
expect(summary).toHaveTextContent(...);
```

Lesson: test visible content while respecting the real DOM structure.

Tests placed in the wrong file

A `ShortAnswerQuestion` test was accidentally added to `OptionQuestion.test.tsx` with an incompatible question model.

Lesson: keep each test suite focused on the component and domain type it owns.

Required callback missing after interface change

After adding `onComplete` to `QuizGameScreenProps`, older renders in tests and `App` no longer satisfied the component contract.

Lesson: TypeScript errors reveal every integration point affected by a public interface change.

localStorage test leakage

Persistent browser state can survive between tests in the same environment.

Lesson: reset external state in `beforeEach` to make tests deterministic.

LF and CRLF warnings

Git reported line-ending normalization warnings on Windows even when file content had not materially changed.

Lesson: separate content changes from platform line-ending metadata before staging a commit.

Architecture lessons

Model invalid states out of the domain

Discriminated unions ensure short-answer questions cannot accidentally require option IDs, and option questions cannot omit their choices.

Keep business rules outside React

Answer validation, normalization, percentage calculation, and storage validation are isolated from UI components.

Store source data, derive secondary data

The application stores:

- Correct answer count
- Total question count
- Completed score records

It derives:

- Percentage
- Current question
- Last-question status
- Correctness display
- Average score
- Personal best

Keep state in the nearest responsible owner

- `OptionQuestion` and `ShortAnswerQuestion` display controlled values
- `QuizGameScreen` owns the active session
- `App` owns completed score history
- `scoreStorage` owns browser persistence operations

Decouple provider data from quiz behavior

The quiz UI consumes the `Quiz` model. It does not know whether that quiz came from fake data, OpenAI, Ollama, or another backend provider.

Senior-level discussion points

Why not use a state-management library?

The state graph remains small and has clear owners. `Context`, `Redux`, or another global store would add indirection without solving a current problem.

Why not persist the active quiz session?

Version 1 only requires completed-score persistence. Persisting partial sessions introduces more state restoration and schema-versioning complexity. It should be added only when product requirements justify it.

Why not grade short answers with an LLM?

Deterministic accepted-answer comparison is faster, cheaper, testable, and predictable. LLM grading may later be useful for semantic answers, but it needs confidence thresholds, cost controls, and fallback behavior.

Why validate `localStorage` data?

Browser storage is external runtime input. It may be corrupted, manually edited, left by an older application version, or contain valid JSON with the wrong structure.

Why use callbacks for completion?

`QuizGameScreen` reports a domain event without owning application persistence. This keeps session logic separate from score storage.

Why does `QuizResultScreen` receive counts rather than answers?

The result screen is presentational. It should not repeat grading logic or inspect the entire quiz session.

Why keep fake data after the local engine works?

Fake data remains a deterministic development and testing provider even after the backend exists. It supports offline development and isolates frontend tests from network failures.

Interview questions and model answers

What is a discriminated union?

A discriminated union is a union of object types that share a literal property, such as `type`, which TypeScript uses to narrow the complete object shape safely.

Why compare option IDs instead of option text?

IDs represent stable identity. Display text can change, be translated, or be duplicated without changing which option is correct.

Why is answer validation a pure utility?

Validation is a domain rule rather than a visual concern. A pure function is reusable, deterministic, and easy to test independently from React.

What is a controlled component?

A controlled component receives its current value through props and reports changes through callback props. Its parent remains the source of truth.

Why does `OptionQuestion` not use `useState`?

`QuizGameScreen` owns the selected option. `OptionQuestion` only displays it and reports interactions.

What causes a React component to render again?

A state update, changed prop, or changed consumed context can schedule another render. In this phase, state setters such as `setSelectedOptionId` and `setCurrentQuestionIndex` drive the quiz UI.

Why use a previous-state callback for the score count?

The new score depends on the previous score. The callback receives the latest value even when React batches updates.

Why is `isCorrect` nullable?

`null` represents an unchecked answer, while `true` and `false` represent checked results. These are three different UI states.

Why lock submitted answers?

Locking prevents changing a response after feedback is visible and protects the integrity of the score.

What is the difference between React state and `localStorage`?

React state is the active in-memory source of truth while the app runs. `localStorage` persists serialized data across reloads and browser sessions.

Why type parsed JSON as `unknown`?

Parsed JSON is untrusted runtime data. `unknown` forces validation before the application treats it as a domain model.

Why use a lazy state initializer?

A lazy initializer loads browser storage only when React creates the state, rather than during every render.

When does `useEffect` run?

It runs after React commits the render. With `[scores]`, it runs after the initial render and after the score collection changes.

Why clear `localStorage` before tests?

External persisted state can leak between tests and make results order-dependent. Clearing it restores deterministic isolation.

Why does `QuizCompletionResult` omit `id` and `completedAt`?

Those are persistence concerns created by `App`. The quiz session only reports the domain result.

What is defensive programming in the submit handler?

The UI disables invalid submission, while the handler independently checks for missing or already-submitted answers. Both layers protect the rule.

Explain-back answers**Why is answer validation outside React?**

Because correctness is a business rule that should remain reusable and independently testable. React components should capture input and display results.

Why do radio inputs share a name but have unique IDs?

The shared name creates one mutually exclusive group. Unique IDs connect each visible label to its specific input.

Why reset all input state between questions?

Each question must start without the previous selection, previous text, or previous feedback.

Why does `App` own completed scores?

`App` coordinates the home screen and quiz screen and is the nearest common owner that needs to update the scoreboard after completion.

Why does the storage utility not update React state?

Its responsibility is browser persistence only. `App` owns the active score state and decides when to update it.

Which is the source of truth while the app is running?

React state is the source of truth. `localStorage` restores and persists that state.

Development commands

Run commands from the repository root:

`C:\CAI\quiz-learning-platform`

Run the frontend

```
npm run dev:web
```

Format all files

```
npm run format
```

Check formatting

```
npm run format:check
```

Run ESLint

```
npm run lint:web
```

Run TypeScript validation

```
npm run typecheck:web
```

Run tests

```
npm run test:web
```

Build the production frontend

```
npm run build:web
```

Run the complete quality gate

```
npm run quality:web
```


Inspect Git changes

```
git status
git diff --stat
git diff
```

Practice exercises

1. Add a second accepted answer to the short-answer fake question and write a test for it.
 2. Add a **Previous question** button without allowing the learner to change already-submitted answers.
 3. Add a visual progress bar derived from the current question index.
 4. Persist only newly completed scores and remove the initial mock scores after the first real completion.
 5. Add a button that clears score history with a confirmation step.
 6. Add storage schema validation for impossible values such as negative answer counts.
 7. Add a test proving an incorrect answer produces a lower final percentage.
 8. Extract a **useQuizSession** hook and explain whether the extraction improves readability.
 9. Add an empty-quiz result and explain how the domain should treat a quiz with zero questions.
 10. Explain how the existing **Quiz** TypeScript model should map to Java records or DTOs.
 11. Replace **fakeQuiz** with a temporary **fetch** call while keeping all question components unchanged.
 12. Add a storage version number and define a migration strategy for old score data.
-

Phase result

At the end of Phase 2, the Quiz Learning Platform has a complete local vertical slice:

```
Configure quiz
-> Play three question types
-> Validate answers
-> Show explanations
-> Navigate questions
-> Calculate score
-> Show result
-> Update scoreboard
-> Persist completed scores
```

The frontend now has a stable quiz contract and tested behavior. The project is ready for Phase 3, which will introduce a Java 21 and Spring Boot backend, a tested health endpoint, Java request and response models, and a fake quiz-generation REST API before OpenAI integration.